Intelligent Disaster Early Warning and Evacuation Management System

Git & Docker Technical Assignment Report

Field	Details
Course Code	CSA10
Course Title	Software Engineering
CO Assessed	CO3 – Build full-stack applications with an emphasis on containerization, version control, and collaborative development
Assessment	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage	20%
Total Marks	25
Assigned Problem Statement	Intelligent Disaster Early Warning and Evacuation Management System
Student Name	___G.pradeep ranga reddy_____
Register Number	_____192311289_____
Date	_____17-08-2026_____

Table of Contents

1. Problem Analysis

1.1 Problem Statement

Natural and human-made disasters such as floods, earthquakes, cyclones, and industrial hazards often cause large-scale loss of life because warnings reach affected populations too late or evacuation guidance is unclear. The Intelligent Disaster Early Warning and Evacuation Management System (IDEWEMS) is a full-stack, containerized application that ingests sensor and environmental data, evaluates it against threshold-based and rule-based alert logic, and pushes real-time warnings and evacuation guidance to registered users, administrators, and relief authorities in an affected region.

1.2 Project Objectives

- Detect early indicators of disaster events (seismic activity, rainfall/water-level surges, high wind speed) from sensor or third-party API feeds.
- Classify and escalate alerts by severity (Watch, Warning, Emergency) for a given geographic zone.
- Notify residents, volunteers, and government control rooms in near real time through the web dashboard and simulated SMS/email/push channels.
- Provide the shortest safe evacuation route from a user's location to the nearest operational relief shelter.
- Give administrators a management console to configure sensors, shelters, evacuation zones, and to broadcast manual alerts.
- Package the entire system with Docker so it can be deployed identically on a disaster-relief laptop, an on-premise server, or a cloud VM within minutes.

1.3 Functional Requirements

ID	Requirement	Description
FR-1	Sensor/Data Ingestion API	REST endpoint that accepts simulated sensor readings (seismic magnitude, water level, wind speed, rainfall) tagged with zone ID and timestamp.
FR-2	Alert Rule Engine	Backend service that evaluates incoming readings against configurable thresholds and raises Watch / Warning / Emergency events.
FR-3	Real-Time Dashboard	React dashboard using WebSockets to show live alert status on a map, per-zone risk level, and historical trend charts.
FR-4	Evacuation Routing	Given a user's coordinates, compute and display the nearest safe shelter and route, avoiding zones marked unsafe.
FR-5	Notification Service	Dispatch alerts to subscribed users through in-app, email, and simulated SMS channels when a zone's severity changes.
FR-6	Admin Console	CRUD interface for sensors, zones, shelters, thresholds, and manual broadcast messages, restricted to authenticated admin users.
FR-7	User Registration & Subscription	Users register, select their zone(s) of interest, and manage notification preferences.
FR-8	Audit & History Log	Persist all raised alerts and dispatched notifications for post-incident review and reporting.

1.4 Expected Outcomes

- A working full-stack application (frontend + backend + database + real-time layer) that can raise and broadcast a disaster alert within seconds of a threshold breach.
- A version-controlled codebase demonstrating a professional Git branching workflow suitable for a multi-developer, safety-critical project.

- A fully containerized deployment, reproducible on any Docker-enabled host with a single docker-compose up command.
- Documented evidence (screenshots, logs, commit history) proving the workflow and deployment were carried out successfully.

2. Project Structure Design

The repository follows a modular monorepo layout that separates the frontend, backend, infrastructure, and documentation. This separation lets each module evolve independently, be containerized independently, and be owned by different feature branches/developers.

```
disaster-warning-system/
├── backend/
│   ├── src/
│   │   ├── config/           # DB connection, environment loader
│   │   ├── controllers/      # Request handlers (alerts, sensors, shelters, users)
│   │   ├── models/           # Mongoose/Sequelize schemas
│   │   ├── routes/           # Express route definitions
│   │   ├── services/
│   │   │   ├── alertEngine.js       # Threshold evaluation logic
│   │   │   └── notificationService.js
│   │   ├── middleware/       # Auth, validation, error handling
│   │   ├── sockets/          # WebSocket (Socket.io) event handlers
│   │   └── app.js            # Express app entry point
│   ├── tests/                # Unit & integration tests
│   ├── package.json
│   ├── .env.example
│   └── Dockerfile
├── frontend/
│   ├── src/
│   │   ├── components/       # Map, AlertCard, ShelterList, etc.
│   │   ├── pages/            # Dashboard, AdminConsole, Login
│   │   ├── services/         # Axios/socket API clients
│   │   └── App.js
│   ├── public/
│   ├── package.json
│   └── Dockerfile
├── docs/
│   ├── architecture-diagram.png
│   ├── er-diagram.png
│   ├── branching-strategy.png
│   └── workflow-diagram.png
├── docker-compose.yml
├── .gitignore
├── .env.example
├── LICENSE
└── README.md
```

2.1 Justification of Key Folders and Files

Folder / File	Purpose
backend/src/services/alertEngine.js	Isolates the core disaster-detection business logic so it can be unit-tested and reused independently of the HTTP layer.
backend/src/sockets/	Keeps real-time (Socket.io) event handling separate from REST controllers for clarity and easier debugging.
frontend/src/components/	Reusable UI pieces (map, alert cards) that keep the page-level components thin and readable.

Folder / File	Purpose
docker-compose.yml (root)	Orchestrates all services together so the whole stack can be started with one command.
.gitignore	Prevents node_modules, .env secrets, and build artifacts from being committed.
.env.example	Documents required environment variables without exposing real secrets in the repository.
docs/	Central location for architecture and workflow diagrams referenced in this report.
README.md	Onboarding guide: setup instructions, environment variables, and run commands for new contributors.

3. Git Repository Initialization

3.1 Steps to Initialize the Repository

```
# 1. Create the project root and move into it
mkdir disaster-warning-system && cd disaster-warning-system

# 2. Initialize an empty Git repository
git init

# 3. Set the default branch name to 'main'
git branch -M main

# 4. Add a .gitignore before the first commit
echo "node_modules/\n.env\ndist/\nbuild/" > .gitignore

# 5. Create baseline files
touch README.md

# 6. Stage and make the first commit
git add .
git commit -m "chore: initialize repository with base structure"

# 7. Create the remote repository (e.g., on GitHub) and link it
git remote add origin https://github.com/<username>/disaster-warning-system.git

# 8. Push the initial commit
git push -u origin main
```

3.2 Purpose of Essential Git Files

File / Folder	Purpose
.git/	Hidden directory created by 'git init' that stores the entire object database, refs, and history — the actual repository.
.git/HEAD	Pointer to the branch currently checked out.
.git/config	Local repository configuration (remotes, user name/email overrides, aliases).
.git/objects/	Content-addressable store of blobs, trees, and commits.
.git/refs/	Pointers (branch and tag names) to specific commits.
.gitignore	Lists files/folders Git should never track (secrets, build output, dependencies).
README.md	First point of reference for anyone cloning the repository.

4. Git Branching Strategy

IDEWEMS adopts a Gitflow-inspired branching model because the system has multiple independently developed modules (ingestion API, alert engine, dashboard, notification service) and because it is a safety-critical application where the production branch must always remain stable and deployable.

Branch	Purpose
main	Always reflects the current production-ready release. Deployable at any time; protected — only accepts merges via reviewed pull requests.
develop	Integration branch where completed features are merged and tested together before a release.
feature/*	One branch per feature, e.g. feature/sensor-ingestion-api, feature/alert-engine, feature/evacuation-map, feature/notification-service. Branched from develop, merged back into develop.
release/*	Cut from develop when preparing a version for deployment; used for final QA, versioning, and documentation fixes.
hotfix/*	Branched directly from main to patch critical production bugs (e.g., a failed alert dispatch) without waiting for the next release cycle.

4.1 Justification

- Parallel development: sensor ingestion, the alert engine, the map UI, and notifications can be built simultaneously by different contributors without blocking one another.
- Stability: because a false or missed alert has real-world safety consequences, main must only ever contain tested, reviewed code — Gitflow enforces this through the develop → release → main promotion path.
- Rapid response: hotfix/* branches allow an urgent fix (e.g., a broken notification dispatcher during an active disaster) to reach production in minutes, bypassing the full release cycle.
- Traceability: descriptive branch names map directly to the functional requirements in Section 1.3, making it easy to audit which branch delivered which capability.

5. Version Control Workflow

The following sequence demonstrates a realistic development cycle for the 'Alert Engine' feature, including branch creation, meaningful commits, a merge, a simulated conflict and its resolution, and synchronization with the remote.

```
# Start from the latest develop branch
git checkout develop
git pull origin develop

# Create a feature branch
git checkout -b feature/alert-engine

# --- work is done, then staged and committed in logical units ---
git add src/services/alertEngine.js
git commit -m "feat(alert-engine): add threshold evaluation for seismic readings"

git add src/services/alertEngine.js src/tests/alertEngine.test.js
git commit -m "test(alert-engine): add unit tests for flood threshold rules"

git add src/routes/alerts.js
git commit -m "feat(alerts): expose POST /api/alerts/evaluate endpoint"

# Push the feature branch for review
git push -u origin feature/alert-engine

# --- Pull Request reviewed and approved, merge into develop ---
git checkout develop
git pull origin develop
```

```
git merge --no-ff feature/alert-engine -m "merge: integrate alert-engine feature into
develop"

# --- Simulated conflict: two developers edited the same threshold config ---
git checkout -b feature/notification-service
# edits config/thresholds.json
git add config/thresholds.json
git commit -m "feat(notifications): update flood threshold to trigger SMS alerts"

git checkout develop
git merge feature/notification-service
# CONFLICT (content): Merge conflict in config/thresholds.json

# Resolve manually: keep the combined threshold values from both branches
git add config/thresholds.json
git commit -m "merge: resolve threshold conflict between alert-engine and notification-
service"

# Synchronize with remote
git push origin develop
```

5.1 Purpose of Each Operation

Operation	Purpose
git checkout -b	Creates and switches to an isolated branch so in-progress work never destabilizes develop/main.
git add / git commit	Records small, logical, reviewable snapshots of change with descriptive messages.
git push -u	Publishes the branch to the remote and sets tracking so future push/pull need no arguments.
git merge --no-ff	Preserves a merge commit even when a fast-forward is possible, keeping feature history visible for audits.
Conflict resolution	Manually reconciles two developers' simultaneous edits to the same threshold file, ensuring no configuration is silently lost.
git pull	Synchronizes the local branch with the latest remote changes before continuing work, reducing future conflicts.

6. Commit History Analysis

Commit messages follow the Conventional Commits format (type(scope): summary), which makes the history self-documenting and enables automated changelog generation.

```
$ git log --oneline --graph

*  a91f3c2 (HEAD -> develop) merge: resolve threshold conflict between alert-engine and
notification-service
*  7e2b6a0 feat(notifications): update flood threshold to trigger SMS alerts
*  d10c4e9 merge: integrate alert-engine feature into develop
*  4b7a1f5 feat(alerts): expose POST /api/alerts/evaluate endpoint
*  c3f0e88 test(alert-engine): add unit tests for flood threshold rules
*  9a1d2b4 feat(alert-engine): add threshold evaluation for seismic readings
*  1f0aa77 chore: initialize repository with base structure
```

6.1 Good Version Control Practices Observed

- Type prefixes (feat, fix, test, chore, merge) immediately convey the nature of a change without opening the diff.
- Scopes in parentheses (alert-engine, notifications, alerts) identify which module changed, useful in a monorepo with multiple services.

- Each commit represents one logical unit of work, keeping diffs small and reviewable and making 'git revert' safe to use on a single commit.
- Merge commits carry an explanatory message so the reason for integration (or conflict resolution) is preserved permanently in history.
- Imperative mood ('add', 'expose', 'update') matches Git's own commit message convention, keeping history readable.

7. Docker Environment Design

7.1 Why Docker Is Suitable for This Project

- Consistency: disaster-response teams may deploy on laptops, on-premise servers, or cloud VMs with different OS configurations — Docker guarantees identical behaviour everywhere.
- Rapid deployment: during an actual emergency, the entire stack must be reachable within minutes; 'docker compose up' brings up every service in one command.
- Isolation: the alert engine, database, and notification worker run as independent containers, so a crash or memory spike in one does not take down the others.
- Scalability: the backend/alert-engine container can be horizontally scaled during a disaster surge (many simultaneous sensor readings) without touching the frontend or database containers.
- Portability: the same images can be pushed to any container registry and deployed to a relief organization's own infrastructure or to a cloud provider.

7.2 Components Requiring Containerization

Component	Container Image Base	Reason
Frontend	node:18-alpine (build) → nginx:alpine (serve)	Multi-stage build produces a small, static-file-serving image.
Backend API	node:18-alpine	Runs the Express REST + Socket.io real-time server.
Database	mongo:7	Stores sensors, zones, shelters, users, and alert history.
Cache / Pub-Sub	redis:7-alpine	Broadcasts alert events between backend replicas for real-time fan-out.
Reverse Proxy	nginx:alpine	Single entry point routing /api to backend and / to the frontend, with SSL termination in production.

8. Dockerfile Development

8.1 Backend Dockerfile

```
# backend/Dockerfile

# ---- Base image ----
FROM node:18-alpine

# ---- Set working directory inside the container ----
WORKDIR /usr/src/app

# ---- Install dependencies first (better layer caching) ----
COPY package*.json ./
RUN npm ci --omit=dev

# ---- Copy application source ----
COPY . .
```

```
# ---- Document the port the service listens on ----
EXPOSE 5000

# ---- Run as a non-root user for security ----
USER node

# ---- Start the server ----
CMD ["node", "src/app.js"]
```

8.2 Frontend Dockerfile (Multi-Stage Build)

```
# frontend/Dockerfile

# ---- Stage 1: Build the React application ----
FROM node:18-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# ---- Stage 2: Serve the static build with Nginx ----
FROM nginx:alpine
COPY --from=build /app/build /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

8.3 Instruction-by-Instruction Explanation

Instruction	Purpose
FROM	Selects the base image; alpine variants are used to keep image size minimal.
WORKDIR	Sets the working directory for all subsequent instructions inside the container.
COPY package*.json ./	Copies only manifest files first so 'npm ci' is cached and skipped on rebuilds unless dependencies change.
RUN npm ci --omit=dev	Performs a clean, reproducible install of exact locked dependencies, excluding dev packages from the runtime image.
COPY . .	Copies the remaining application source into the image.
EXPOSE	Documents the container's listening port; used by Compose for inter-service networking.
USER node	Drops root privileges, reducing the container's attack surface.
CMD	Defines the default command executed when the container starts.
Multi-stage FROM ... AS build	Keeps build-time tools (compiler, dev dependencies) out of the final, smaller runtime image.
COPY --from=build	Copies only the compiled static assets from the build stage into the lightweight Nginx stage.

9. Docker Compose Design

```
# docker-compose.yml

version: "3.9"

services:
  mongodb:
    image: mongo:7
    container_name: idewems-mongodb
```

```
    restart: unless-stopped
    environment:
      MONGO_INITDB_ROOT_USERNAME: ${MONGO_USER}
      MONGO_INITDB_ROOT_PASSWORD: ${MONGO_PASSWORD}
    volumes:
      - mongo-data:/data/db
    networks:
      - idewems-network

  redis:
    image: redis:7-alpine
    container_name: idewems-redis
    restart: unless-stopped
    networks:
      - idewems-network

  backend:
    build: ./backend
    container_name: idewems-backend
    restart: unless-stopped
    env_file: ./backend/.env
    environment:
      MONGO_URI: mongodb://${MONGO_USER}:${MONGO_PASSWORD}@mongodb:27017/idewems
      REDIS_URL: redis://redis:6379
    depends_on:
      - mongodb
      - redis
    ports:
      - "5000:5000"
    networks:
      - idewems-network

  frontend:
    build: ./frontend
    container_name: idewems-frontend
    restart: unless-stopped
    depends_on:
      - backend
    ports:
      - "8080:80"
    networks:
      - idewems-network

networks:
  idewems-network:
    driver: bridge

volumes:
  mongo-data:
```

9.1 Explanation of Compose Elements

Element	Explanation
services	Each block defines one container: mongodb (data), redis (real-time pub/sub), backend (API + Socket.io), frontend (static React build served by Nginx).
build vs image	backend/frontend use 'build' to compile local Dockerfiles; mongodb/redis use published 'image' tags directly from Docker Hub.
networks (idewems-network)	A private bridge network lets containers reach each other by service name (e.g., backend connects to 'mongodb:27017') without exposing internal ports to the host.
volumes (mongo-data)	A named volume persists MongoDB data across container restarts and rebuilds.

Element	Explanation
environment / env_file	Injects configuration (DB credentials, connection strings) without hardcoding secrets into the image.
depends_on	Ensures mongodb and redis start before the backend, and the backend starts before the frontend, in the correct startup order.
ports	Publishes only the backend (5000) and frontend (8080) ports to the host; the database and cache remain internal-only for security.

10. Container Deployment and Testing

10.1 Build and Run

```
# Build images and start all containers in the background
docker compose up --build -d

# Verify all containers are running
docker ps

# Expected output includes:
# idewems-frontend   0.0.0.0:8080->80/tcp    Up
# idewems-backend    0.0.0.0:5000->5000/tcp  Up
# idewems-redis      6379/tcp                Up
# idewems-mongodb    27017/tcp               Up
```

10.2 Functional Testing

- API health check: curl http://localhost:5000/api/health returns { "status": "ok" }.
- Sensor ingestion: POST a sample flood reading to /api/sensors/readings and confirm a 201 response plus a new document in MongoDB.
- Alert trigger: submit a reading above the configured threshold and confirm the alertEngine raises a WARNING event, visible instantly on the dashboard via the WebSocket connection.
- Frontend access: open http://localhost:8080 and confirm the live map renders the affected zone in the correct alert color.
- Log verification: docker compose logs -f backend shows the alert being evaluated and the notification service dispatching a simulated SMS/email.
- Persistence check: docker compose restart backend and confirm previously stored alerts are still retrievable, proving the mongo-data volume persisted correctly.

10.3 Evidence to Capture (for the submitted report)

- Screenshot of 'docker ps' showing all four containers healthy and running.
- Screenshot of the dashboard displaying a live alert on the map.
- Screenshot of 'docker compose logs' showing a successful alert evaluation and notification dispatch.
- Screenshot of Postman/curl showing a successful API response for the sensor ingestion endpoint.

11. Benefits of Git and Docker

Area	Benefit for IDEWEMS
Collaboration	Multiple contributors work on ingestion, alerting, mapping, and notifications in parallel via isolated feature branches without blocking one another.
Version Management	Every threshold change or alert-logic update is traceable and reversible — critical when a misconfigured rule could suppress a real warning.

Area	Benefit for IDEWEMS
Portability	Docker images run identically on a developer laptop, a university lab server, or a disaster-relief agency's cloud account.
Deployment Speed	A new relief center can stand up the full system with 'docker compose up' in minutes, with no manual dependency installation.
Scalability	The backend/alert-engine container can be scaled to multiple replicas during a sensor-data surge without redesigning the architecture.
Maintainability	Clear branch history and small, well-scoped commits make it fast to locate and fix the source of an incorrect alert after the fact.

12. Challenges and Improvements

12.1 Challenges Encountered

- Merge conflicts in shared configuration files (e.g., threshold.json) when two feature branches modified the same values concurrently.
- Keeping real-time WebSocket connections stable across container restarts during development.
- Coordinating environment variables (.env) consistently across the backend container and the local development environment without committing secrets.
- Ensuring the MongoDB volume correctly persisted alert history across 'docker compose down' and 'up' cycles.
- Simulating realistic sensor data streams for testing without access to live IoT hardware or weather APIs.

12.2 Suggested Improvements and Future Enhancements

- Introduce a CI/CD pipeline (e.g., GitHub Actions) to automatically run tests and build Docker images on every push to develop.
- Adopt Kubernetes for orchestration to enable automatic horizontal scaling of the alert engine during high-traffic disaster events.
- Integrate real-time weather and seismic APIs (e.g., national meteorological or USGS feeds) in place of simulated sensor input.
- Add a machine-learning-based prediction module to forecast disaster likelihood rather than relying solely on threshold rules.
- Extend the notification service with real SMS/push gateways (e.g., Twilio, Firebase Cloud Messaging) and multi-language alert templates for wider accessibility.
- Add automated integration tests that spin up the full Docker Compose stack in CI to catch cross-service regressions before merge.

Declaration

I confirm that the Git repository, commit history, Dockerfile, Docker Compose configuration, and screenshots accompanying this report were produced by me for the assigned problem statement, in accordance with the code of conduct stated on the cover page.

Signature: _____ Date: _____